Phase 1: Foundations

1. Class

Blueprint/template for creating objects.

2. Object

Real instance of a class containing actual data.

3. Attributes (Instance Variables)

Data stored inside objects.

4. Methods

Actions/behaviors performed by objects.

5. Constructors

Special methods used to initialize objects.

6. Parameterized Constructors

Constructors that accept values during object creation.

7. `this` Keyword

Refers to the current object.

Phase 2: Core OOP Pillars

8. Encapsulation

Hide data using `private` and provide controlled access through methods.

9. Inheritance

Reuse code using parent-child relationships (`extends`).

10. Polymorphism

One interface, many forms.

a. Method Overloading

Same method name, different parameters.

b. Method Overriding

Child class provides its own implementation.

11. Abstraction

Hide implementation details and show only essential functionality.

12. Interfaces

Define contracts that classes must implement.

Phase 3: Relationships Between Classes

13. IS-A Relationship

Used in inheritance.

Example:

```
Doctor IS-A Employee
```

14. HAS-A Relationship

One class contains another class.

Example:

```
Hospital HAS-A Doctor
Patient HAS-A MedicalRecord
```

15. Association

General relationship between objects.

Example:

```
Doctor treats Patient
```

16. Aggregation

Weak HAS-A relationship.

Example:

```
Hospital HAS-A Doctor
```

Doctor can exist without Hospital.

17. Composition

Strong HAS-A relationship.

Example:

```
Patient HAS-A MedicalRecord
```

MedicalRecord cannot exist without Patient.

Phase 4: Object Class & Utility Topics

18. Object Class

Parent of all Java classes.

19. toString()

Convert object to readable string.

20. equals()

Compare object contents.

21. hashCode()

Used in hashing collections.

22. instanceof

Check object type.

Phase 5: Advanced Constructor Topics

23. Constructor Overloading

Multiple constructors with different parameters.

24. Constructor Chaining

Calling one constructor from another.

25. `super()` Constructor

Call parent class constructor.

Phase 6: Advanced Inheritance Topics

26. Single Inheritance

One parent, one child.

27. Multilevel Inheritance

Grandparent → Parent → Child.

28. Hierarchical Inheritance

One parent, multiple children.

29. Why Java Doesn't Support Multiple Class Inheritance

Diamond problem discussion.

Phase 7: Static & Final

30. Static Variables

Shared among all objects.

31. Static Methods

Belong to class, not object.

32. Static Blocks

Run when class loads.

33. Final Variables

Constants.

34. Final Methods

Cannot be overridden.

35. Final Classes

Cannot be inherited.

Phase 8: Abstract Classes & Interfaces

36. Abstract Class

Partial abstraction.

37. Abstract Methods

Methods without implementation.

38. Interface

100% contract-based design.

39. Multiple Inheritance Through Interfaces

Java's solution to multiple inheritance.

40. Default Methods

Methods with implementation inside interfaces.

Phase 9: Runtime Concepts

41. Upcasting

```
Employee e = new Doctor();
```

42. Downcasting

```
Doctor d = (Doctor)e;
```

43. Dynamic Method Dispatch

Runtime method overriding.

Phase 10: SOLID Foundations

44. Single Responsibility Principle (SRP)

45. Open Closed Principle (OCP)

46. Liskov Substitution Principle (LSP)

47. Interface Segregation Principle (ISP)

48. Dependency Inversion Principle (DIP)

Phase 11: Design Patterns (After OOP)

49. Singleton

50. Factory Method

51. Builder Pattern

52. Strategy Pattern

53. Observer Pattern

54. Adapter Pattern

55. MVC Pattern

Recommended Learning Order

For your Hospital Management System:

```
1. Class
2. Object
3. Attributes
4. Methods
5. Constructor
6. this Keyword
7. Encapsulation
8. Inheritance
9. Method Overloading
10. Method Overriding
11. Polymorphism
12. Abstraction
13. Interface
14. HAS-A Relationship
15. Aggregation
16. Composition
17. Static
18. Final
19. Object Class
20. SOLID Basics
```